

Modular SOC Decomposition Framework

Architecture Documentation for Multi-Model Intercomparison

Lorenzo — HIKET Project

2026-02-26

1. Overview

This document defines a modular architecture for soil organic carbon (SOC) decomposition modelling within the HIKET multi-model intercomparison framework. The goal is to define all models — Yasso07, Yasso15, Yasso20, RothC, and Q-model — through common functional interfaces that cleanly separates **litter input processing**, **environmental modification of kinetics**, and **decomposition dynamics**.

This separation enables modularity.

2. General Framework

2.1 Core Equation

The simulated soil organic carbon over time is expressed as:

$$\mathbf{C}(t, k) = f_{d_k} \left(f_{i_k}(\mathbf{I}(t)), \xi_k(t) \right)$$

where:

Symbol	Description	Dimensions
$\mathbf{C}(t)$	SOC pool vector at time t	$[n_{\text{pools}} \times 1]$
$\mathbf{I}(t)$	Raw litter input data at time t	model-agnostic format
$f_i(\cdot)$	Input compatibility layer — transforms raw inputs into model-specific pool inputs	$\mathbf{I}(t) \rightarrow \mathbf{J}(t)$
$f_d(\cdot)$	Decomposition function — the structural core of each model	$(\mathbf{J}(t), \xi(t)) \rightarrow \mathbf{C}(t)$
$\xi(t)$	Kinetic modifier — composite environmental scalar(s)	$\in \mathbb{R}^{n_\xi}$

The k denotes model-specific functions.

The kinetic modifier is itself composed of independent environmental response functions:

$$\xi(t) = \xi_T(T(t)) \cdot \xi_M(M(t)) \cdot \xi_e$$

where ξ_T is the temperature modifier, ξ_M is the moisture/precipitation modifier, and ξ_e represents any additional static or time-varying modifiers, likely edaphic (e.g., clay content, soil cover).

2.2 Architecture Schema

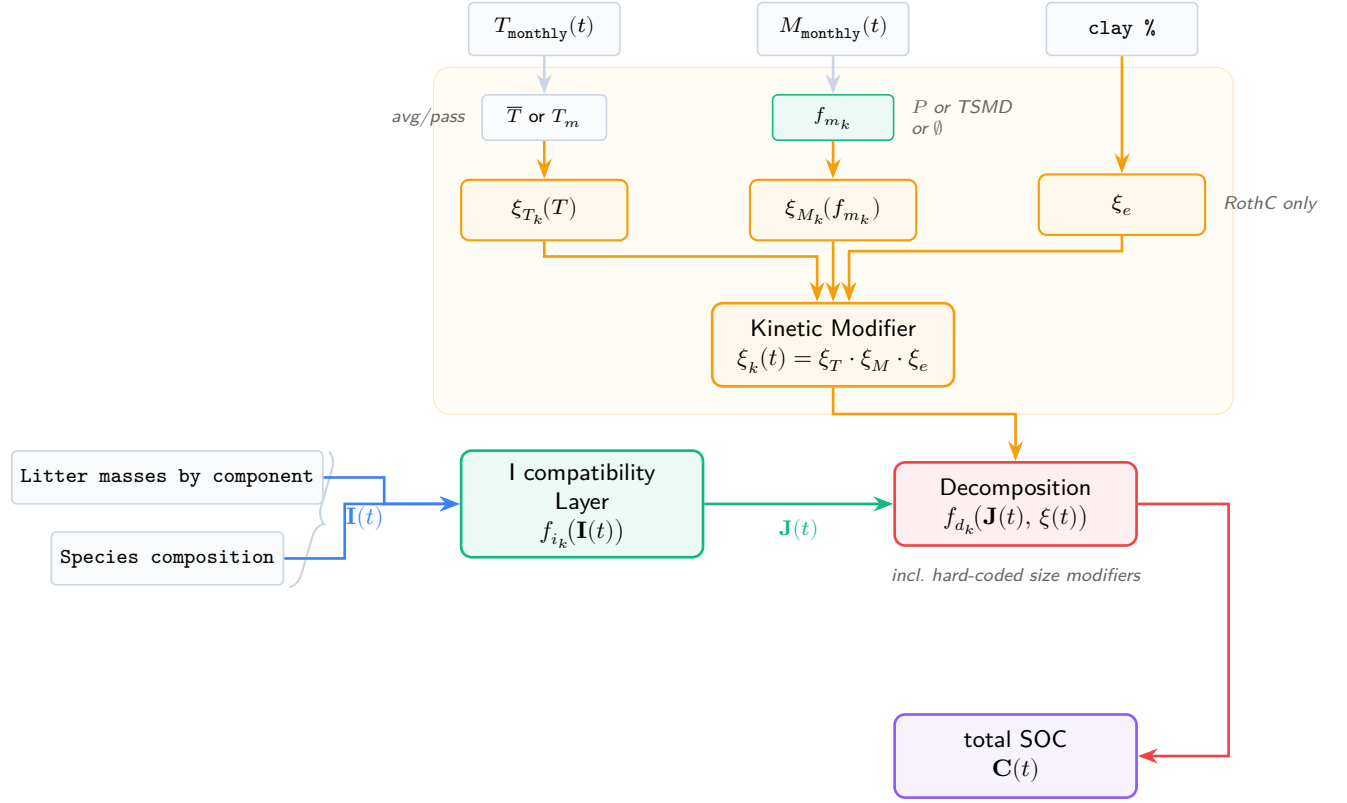


Figure 1: Modular architecture of the SOC decomposition framework. Raw inputs $\mathbf{I}(t)$ are transformed by the input compatibility layer f_i into model-specific inputs $\mathbf{J}(t)$. Environmental drivers are combined into the composite kinetic modifier $\xi(t)$. Both feed into the model-specific decomposition function f_d .

3. Input requirements

3.1 Standardised input format — $\mathbf{I}(t)$

Dimensions $N_{plots} \times N_{timesteps}$

The standard input data for all models $\mathbf{I}(t)$ contains litter mass inputs from six cohorts (foliage, fine roots, branches, coarse roots, stump, stem). The species group (pine, spruce, birch) are partitioned based on species composition data, which are assumed constant per plot.

3.2 Standardised environmental drivers

3.2.1 Time-variant drivers

Dimensions $N_{plots} \times N_{timesteps}$

Temperature and moisture inputs are provided as monthly time series. Temperature relies only on a compatibility layer to aggregate to annual summaries where needed. Moisture is more complicated since the models rely on different inputs (precipitation vs topsoil moisture deficit), so we need a compatibility layer to convert precipitation to soil moisture deficit for RothC, and to pass precipitation directly to Yasso. The Q-model does not explicitly include a moisture modifier, but implicitly via litter quality.

3.2.2 Static edaphic modifiers

Dimensions $N_{plots} \times 1$

Clay content is provided as a scalar per plot, as well as tree species composition.

3. Output structure

3.1 Model-specific output

Dimensions $N_{plots} \times N_{timesteps} \times N_{pools}$, replicated for each model (different data frames for each model, with different pool names and numbers).

3.1 Total SOC output

Dimensions $N_{plots} \times N_{timesteps} \times N_{models}$

3. Components requirements

3.1 Input Compatibility Layer — $f_i(\mathbf{I}(t))$

The input layer f_i maps this standardised input into the model-specific input vector $\mathbf{J}(t)$:

$$f_i : \mathbf{I}(t) \longrightarrow \mathbf{J}(t) \in \mathbb{R}^{n_{\text{input pools}}}$$

The mapping differs across models:

Model	Input pools $\mathbf{J}(t)$	Key transformation
Yasso07/15/20	AWEN fractions (A, W, E, N)	Total mass to AWEN via component-specific coefficients
RothC	DPM, RPM, BIO, HUM, IOM	Total mass $\rightarrow$ DPM/RPM split via component-dependent coefficients

Model	Input pools $\mathbf{J}(t)$	Key transformation
Q-model	Quality-distributed cohorts per tissue type	Separate cohorts retained; quality q_0 assigned per species and component

3.2 Kinetic Modifier — $\xi(t)$

The kinetic modifier captures all environmental effects on decomposition rates. It is composed multiplicatively:

$$\xi(t) = \xi_T(T(t)) \times \xi_M(f_{m_k}(M(t))) \times \xi_k$$

Each sub-modifier is a vector over time (or, for edaphic effects ξ_k , a scalar, constant over time) that rescales the baseline decomposition rates.

3.2.1 Temperature Modifier — ξ_T

All functions accept the same input, with the only compatibility layer being a monthly to annual averaging.

The functional form of the temperature response differs across models:

Model	Form	Expression
Yasso07/15	Gaussian-like, 4-point annual	$\xi_T = \frac{1}{4} \sum_{i=1}^4 \exp(\beta_1 T_i + \beta_2 T_i^2)$, T_i from annual mean + amplitude
Yasso20	Gaussian-like, exact monthly	$\xi_T = \frac{1}{12} \sum_{m=1}^{12} \exp(\beta_1 T_m + \beta_2 T_m^2)$, T_m are observed monthly values
RothC	Arrhenius-based	$\xi_T = \frac{47.9}{1 + \exp(\frac{106}{T+18.3})}$
Q-model	Linear	$\xi_T = \max(0, a_T + b_T \cdot T)$

3.2.2 Moisture/Precipitation Modifier — ξ_M

Moisture functions of different models accept different inputs, so we need another compatibility layer, f_{m_k} . Yasso uses precipitation, RothC uses topsoil moisture deficit, and Q-model does not explicitly include a moisture modifier (but implicitly via litter quality).

Model	Driver	Notes
Yasso07/15	Annual precipitation P	$\xi_M = 1 - \exp(\gamma \cdot P/1000)$; pool-group-specific for Yasso15
Yasso20	Monthly precipitation (summed annually)	Same formula; P derived internally as annual sum of monthly values
RothC	Topsoil moisture deficit	Rate-modifying factor from TSMD accumulation
Q-model	—	Not explicitly included (implicitly via litter quality)

3.2.3 Additional Static Modifiers — ξ_k

Modifier	Applicable models	Description
Clay (ξ_{clay})	RothC	Modifies BIO/HUM partitioning: $x = 1.67(1.85 + 1.60 \exp(-0.0786 \% \text{clay}))$
Soil cover	RothC	Binary modifier on decomposition rate for bare soil, WON'T BE IMPLEMENTED!

3.2.4 Hard-coded Static Modifiers

Both Q and Yasso have static modifiers dependent on size that are specific to litter classes. These are hard-coded in each model function, rather than passed as separate modifiers.

For the Yasso family of models, all size modifiers are hard-coded in the R wrappers

4. Decomposition Equations

This section presents the complete decomposition dynamics for each model

4.1 Yasso07

Pool structure

Five pools: $\mathbf{C} = [C_A, C_W, C_E, C_N, C_H]^T$, representing Acid-soluble, Water-soluble, Ethanol-soluble, Non-soluble (lignin-like), and Humus.

Dynamics

$$\frac{d\mathbf{C}}{dt} = \mathbf{A} \mathbf{C} + \mathbf{I}$$

where $\mathbf{I} = [I_A, I_W, I_E, I_N, 0]^T$ (no direct input to Humus).

Diagonal elements (loss from pool i):

$$A_{ii} = -\alpha_i \cdot \xi_T \cdot \xi_M$$

where α_i is the base decomposition rate. For the N pool, an additional size modifier applies to woody litter:

$$A_{44} = -\alpha_N \cdot \xi_T \cdot \xi_M \cdot f_{\text{size}}$$

with $f_{\text{size}} = \min(1, \exp(-\delta_1 \cdot d^{\delta_2} \cdot |r|))$, where d is the wood diameter (cm).

Off-diagonal elements (transfer from pool j to pool i):

$$A_{ij} = p_{ij} \cdot |\alpha_j \cdot \xi_T \cdot \xi_M|$$

Humus formation (row 5, columns 1–4):

$$A_{5j} = p_H \cdot |\alpha_j \cdot \xi_T \cdot \xi_M| \quad \text{for } j \in \{A, W, E, N\}$$

Respiration is the residual: $R_h = \sum_j |A_{jj}| C_j - \sum_{i \neq j} A_{ij} C_j$.

Environmental modifiers

Temperature — four-point approximation of intra-annual variation:

$$\xi_T = \frac{1}{4} \sum_{i=1}^4 \exp(\beta_1 T_i + \beta_2 T_i^2)$$

where the quarterly temperatures are derived from annual mean $\bar{T}$ and amplitude A_T :

$$\begin{aligned} T_1 &= \bar{T} + \frac{4A_T}{\pi} \left(\frac{1}{\sqrt{2}} - 1 \right), & T_2 &= \bar{T} - \frac{4A_T}{\sqrt{2}\pi} \\ T_3 &= \bar{T} + \frac{4A_T}{\pi} \left(1 - \frac{1}{\sqrt{2}} \right), & T_4 &= \bar{T} + \frac{4A_T}{\sqrt{2}\pi} \end{aligned}$$

Precipitation:

$$\xi_M = 1 - \exp\left(\gamma \cdot \frac{P}{1000}\right)$$

where P is annual precipitation (mm).

Analytical solution

For annual timestep $\Delta t = 1, \text{yr}$:

$$\mathbf{C}(t+1) = e^{\mathbf{A}} \cdot \mathbf{C}(t) + (e^{\mathbf{A}} - \mathbf{I}_5) \mathbf{A}^{-1} \mathbf{I}$$

where $e^{\mathbf{A}}$ is the matrix exponential.

Steady state:

$$\mathbf{C}^* = -\mathbf{A}^{-1} \mathbf{I}$$

Default parameters

Parameter	Value	Description
$\alpha_A, \alpha_W, \alpha_E, \alpha_N$	0.66, 4.30, 0.35, 0.22	Base decomposition rates (yr^{-1})
α_H	0.0033	Humus decomposition rate (yr^{-1})
p_H	0.04	Fraction to Humus
β_1, β_2	0.096, \$-\$0.0014	Temperature response parameters
γ	\$-\$1.21	Precipitation response parameter
δ_1, δ_2, r	\$-\$1.7, 0.86, \$-\$0.306	Size modifier parameters

4.2 Yasso15

Yasso15 retains the AWENH pool structure and transfer matrix of Yasso07, but introduces two structural changes.

Pool-group-specific climate responses. Where Yasso07 applies a single temperature and precipitation modifier to all pools, Yasso15 separates the climate response into three groups — AWE, N, and H — each with independent parameters:

$$\xi_T^{\text{AWE}} = \frac{1}{4} \sum_{i=1}^4 \exp(\beta_1 T_i + \beta_2 T_i^2), \quad \xi_T^N = \frac{1}{4} \sum_{i=1}^4 \exp(\beta_{N1} T_i + \beta_{N2} T_i^2), \quad \xi_T^H = \frac{1}{4} \sum_{i=1}^4 \exp(\beta_{H1} T_i + \beta_{H2} T_i^2)$$

Similarly for precipitation:

$$\xi_M^{\text{AWE}} = 1 - \exp(\gamma \cdot P/1000), \quad \xi_M^N = 1 - \exp(\gamma_N \cdot P/1000), \quad \xi_M^H = 1 - \exp(\gamma_H \cdot P/1000)$$

The combined climate modifier is applied per pool group when constructing the diagonal of $\mathbf{A}$:

$$\begin{aligned} A_{ii} &= -\alpha_i \cdot \xi_T^{\text{AWE}} \cdot \xi_M^{\text{AWE}} \cdot f_{\text{size}} && \text{for } i \in \{A, W, E\} \\ A_{44} &= -\alpha_N \cdot \xi_T^N \cdot \xi_M^N \cdot f_{\text{size}} \\ A_{55} &= -\alpha_H \cdot \xi_T^H \cdot \xi_M^H && \text{(no size effect)} \end{aligned}$$

Off-diagonal elements and humus formation follow the same structure as Yasso07, with transfers proportional to the source pool's modified rate.

Leaching. Yasso15 adds a precipitation-dependent leaching term to the AWEN pools (not H):

$$A_{ii} += w \cdot P/1000 \quad \text{for } i \in \{A, W, E, N\}$$

The size modifier, transfer matrix structure, analytical solution (matrix exponential), and steady-state formula ($\mathbf{C}^* = -\mathbf{A}^{-1}\mathbf{b}$) are identical to Yasso07.

4.3 Yasso20

Yasso20 shares the AWENH pool structure, pool-group-specific climate responses, and leaching of Yasso15. The structural equations are the same. The principal differences are in parameterisation and the size modifier formula.

Modified size dependence. Yasso07/15 use an exponential form:

$$f_{\text{size}}^{07/15} = \min(1, \exp(-\delta_1 \cdot d^{\delta_2} \cdot |r|))$$

Yasso20 uses a quadratic formulation:

$$f_{\text{size}}^{20} = (1 + \theta_1 d + \theta_2 d^2)^{-|r|}$$

which produces a stronger size-dependent slowdown for coarse woody debris.

Substantially different parameter values. The most consequential changes from Yasso07/15 are:

Parameter	Yasso07	Yasso20	Change	Effect
p_H	0.04	0.0042	\$-\$90%	Much less humus formation
α_H	0.0033	0.0015	\$-\$54%	Slower humus decomposition
α_W	4.30	5.19	\$+\$21%	Faster W-pool turnover
α_E	0.35	0.13	\$-\$63%	Slower E-pool turnover
α_N	0.22	0.10	\$-\$55%	Slower N-pool turnover

The dramatically reduced p_H ($9.5\times$ less humus formation) is the primary driver of Yasso20 predicting \$20% lower total SOC than Yasso07/15 for Finnish forests, despite the compensating effect of slower humus decomposition.

4.4 RothC

Pool structure

Five pools: $\mathbf{C} = [C_{\text{DPM}}, C_{\text{RPM}}, C_{\text{BIO}}, C_{\text{HUM}}, C_{\text{IOM}}]^T$, representing Decomposable Plant Material, Resistant Plant Material, Microbial Biomass, Humified Organic Matter, and Inert Organic Matter.

Dynamics

RothC operates on a **monthly timestep** $\Delta t = 1/12, \text{yr}$. For each active pool $i \in \{\text{DPM}, \text{RPM}, \text{BIO}, \text{HUM}\}$:

$$C_i(t + \Delta t) = C_i(t) \cdot \exp(-k_i \cdot r(t) \cdot \Delta t) + I_i \cdot \Delta t + \text{transfers}_i$$

where $r(t) = \xi_T(t) \cdot \xi_M(t) \cdot \xi_V$ is the composite rate modifier (normalised so that $\xi_T/47.9$ gives the fractional effect).

The IOM pool is inert: $C_{\text{IOM}}(t) = 0.049 \cdot (\% \text{clay})^{1.139}$.

Decomposed material from each pool:

$$D_i = C_i \cdot (1 - \exp(-k_i \cdot r \cdot \Delta t))$$

This is partitioned into three fluxes governed by the clay-dependent ratio x :

$$x = 1.67(1.85 + 1.60 \exp(-0.0786 \cdot \% \text{clay}))$$

$$\begin{aligned}
f_{\text{CO}_2} &= x/(x+1) \\
f_{\text{BIO}} &= 0.46/(x+1) \\
f_{\text{HUM}} &= 0.54/(x+1)
\end{aligned}$$

So the transfers are:

$$\begin{aligned}
\text{CO}_2 &= \sum_i D_i \cdot f_{\text{CO}_2} \\
C_{\text{BIO}} &+= \sum_i D_i \cdot f_{\text{BIO}} \\
C_{\text{HUM}} &+= \sum_i D_i \cdot f_{\text{HUM}}
\end{aligned}$$

Only DPM and RPM receive external inputs: I_{DPM} , I_{RPM} (partitioned from total litter via the DPM/RPM ratio).

Environmental modifiers

Temperature (monthly):

$$\xi_T = \frac{47.9}{1 + \exp(106 / (T + 18.3))}$$

Moisture — based on accumulated topsoil moisture deficit (TSMD):

$$\text{TSMD}_m = \max(0, E_m - P_m)$$

The moisture modifier reduces decomposition when soil dries:

$$\xi_M = \max(0.2, 1 - \text{TSMD}/\text{TSMD}_{\max})$$

where $\text{TSMD}_{\max}$ depends on clay content.

Vegetation cover:

$$\xi_V = \begin{cases} 0.6 & \text{vegetated (all forest plots)} \\ 1.0 & \text{bare soil} \end{cases}$$

Analytical steady state

The system can be expressed as $\mathbf{A} \mathbf{C}^* = -\mathbf{b}$ for the four active pools, where $\mathbf{A}$ is the 4×4 matrix encoding losses and transfers, and $\mathbf{b} = [I_{\text{DPM}}, I_{\text{RPM}}, 0, 0]^T$.

$$\mathbf{C}_{\text{active}}^* = -\mathbf{A}^{-1} \mathbf{b}$$

The full steady state appends C_{IOM} calculated from clay content.

Default parameters

Parameter	Value	Description
k_{DPM}	10.0	DPM decomposition rate (yr^{-1})
k_{RPM}	0.3	RPM decomposition rate (yr^{-1})

Parameter	Value	Description
k_{BIO}	0.66	BIO decomposition rate (yr^{-1})
k_{HUM}	0.02	HUM decomposition rate (yr^{-1})

4.5 Q-model

Pool structure

Unlike the discrete-pool models above, the Q-model (Bosatta & Ågren, 1985) describes decomposition along a **continuous quality axis** q . Organic matter is characterised by its current quality $q \in (0, q_0]$, where q_0 is the initial quality at litter input. As material decomposes, quality declines and decomposition slows — encoding the universal observation that fresh litter decomposes faster than aged organic matter.

In practice, the model is discretised into **cohort tracking**: each annual litter input forms a cohort whose remaining mass is tracked as a function of age and tissue type.

Five tissue classes are tracked separately, distinguished by initial quality and entry dynamics:

Tissue class	Quality parameter	Entry dynamics
Foliage (needles)	q_{0n} (species-specific)	Immediate
Fine roots	q_{0f} (species-specific)	Immediate
Understorey	q_{0n} (as foliage)	Immediate
Branches	q_{0w} (species-specific)	Gradual fragmentation ($t_{\max,b} = 13$ yr)
Stems	q_{0w} (species-specific)	Gradual fragmentation ($t_{\max,s} = 60$ yr)

Core equations

Microbial uptake rate as a function of quality:

$$u(q) = u_0 \cdot q^\beta$$

where u_0 is the temperature-dependent base rate and β is the quality-rate exponent (default $\beta = 7$).

Temperature dependence (linear form):

$$u_0(T) = u_{00} + u_{01} \cdot T$$

where T is the mean annual temperature (°C), $u_{00} = 0.0855$, and $u_{01} = 0.0157$.

An alternative thermodynamic (Arrhenius) formulation (not implemented here) exists (Bosatta & Ågren 1999):

$$u(q, T) = u_0^{\text{ref}} \cdot \exp\left(\frac{|\Delta G^0|}{Rq} \left(\frac{1}{T_{\text{ref}}} - \frac{1}{T}\right)\right)$$

where T is in Kelvin, $R = 8.314, \text{J, mol}^{-1}, \text{K}^{-1}$, and ΔG^0 is the free energy change per enzymatic step. This formulation implies that low-quality substrates have stronger temperature sensitivity than high-quality substrates — a feature not captured by the linear form.

Derived parameters. The quality decline exponent z controls how decomposition slows over time:

$$z = \frac{1 - e_0}{\beta \cdot \eta_{11} \cdot e_0}$$

where e_0 is the initial microbial growth efficiency (0.25) and η_{11} is the carbon use efficiency parameter (0.36). With default parameters, $z \approx 1.19$.

The composite decomposition rate α for a given tissue type combines all factors:

$$\alpha = f_C \cdot \beta \cdot \eta_{11} \cdot u_0 \cdot q_0^\beta$$

where $f_C = 0.5$ is the carbon fraction of organic matter.

Remaining fraction functions

Simple tissues (foliage, fine roots, understorey) enter the soil immediately and follow power-law decay:

$$g(t) = \frac{1}{(1 + \alpha t)^z}$$

Branches fragment gradually over $t_{\max,b}$ years. The remaining fraction integrates decomposition over the fragmenting material. Defining $i_b = \min(t, t_{\max,b})$:

$$g_b(t) = \frac{2 \left[(1 + \alpha t)^{1-z} - (1 + \alpha(t - i_b))^{1-z} (1 - i_b/t_{\max,b}) \right]}{\alpha (1 - z) t_{\max,b}} + \frac{2 \left[(1 + \alpha(t - i_b))^{2-z} - (1 + \alpha t)^{2-z} \right]}{\alpha^2 (1 - z)(2 - z) t_{\max,b}^2} + \left(1 - \frac{i_b}{t_{\max,b}} \right)^2$$

Stems follow the same structure with $t_{\max,s}$ replacing $t_{\max,b}$.

Quality evolution

Quality declines with mass loss:

$$q(t) = q_0 \left(\frac{C(t)}{C(0)} \right)^{1/z}$$

This couples quality and mass: as carbon is lost, the remaining material becomes progressively more recalcitrant.

Transient temperature implementation

The standard Q-model analytical solution assumes constant α (i.e. constant temperature). Under transient climate, we track **cumulative decomposition potential** for each cohort:

$$\tilde{\alpha}(t) = \sum_{\tau=1}^t \alpha(T(\tau))$$

where $\alpha(T(\tau))$ is recalculated each year from the current temperature. The remaining fraction then becomes:

$$g(t) = \frac{1}{(1 + \tilde{\alpha}(t))^{\bar{z}}}$$

This ensures all carbon pools respond to current temperature conditions rather than being locked to the temperature at the time of input — a critical requirement for climate change projections. Without this correction, the model underestimates SOC loss under warming by 20–40%.

Initialisation: hybrid approach

Rather than running a 1500-year numerical spinup, we use an analytical-hybrid approach:

1. **Analytical steady state.** For simple tissues:

$$C_{ss} = L \cdot \sum_{t=1}^{T_{\max}} g(t)$$

where L is the annual litter input. Branches and stems use numerical integration of their respective g_b , g_s functions over 500–1000 years.

2. **Bulk pool initialisation.** The total C_{ss} is treated as a single bulk pool that decomposes via:

$$C_{init}(t) = C_{ss} \cdot (1 + \alpha_0 t)^{1-z}$$

3. **Short spinup** (10–20 years) builds up explicit cohort matrices while the bulk pool decays. Total SOC = bulk pool + new cohorts.

This reduces initialisation time from \$ \$1500 years to 10–20 years while maintaining accuracy.

Default parameters

Parameter	Value	Description
β	7	Quality–rate exponent
e_0	0.25	Initial microbial growth efficiency
η_{11}	0.36	Carbon use efficiency parameter
f_C	0.5	Carbon fraction of organic matter
u_{00}	0.0855	Base uptake rate at 0°C
u_{01}	0.0157	Temperature sensitivity of uptake
$t_{max,b}$	13	Branch fragmentation time (yr)
$t_{max,s}$	60	Stem fragmentation time (yr)
q_{0n}	1.01 (spruce)	Initial quality, foliage
q_{0f}	1.036	Initial quality, fine roots
q_{0w}	1.00	Initial quality, woody tissues

4.6 Common Interface and Implementation Architecture

All five decomposition models share a common calling interface, enabling the simulation loop and Bayesian calibration framework to treat them interchangeably. The implementation is split across two languages: performance-critical numerical code in **Fortran**, and data handling, calibration logic, and input preparation in **R**.

Fortran–R boundary

The guiding principle is to minimise R↔Fortran call overhead during MCMC sampling. Since the calibration loop evaluates the likelihood millions of times across \$ \$3,700 plots, each evaluation should require a **single Fortran call per plot** that runs the full time-series simulation, rather than hundreds of per-timestep round-trips.


```
resp_out = double(n_years)          # output: annual respiration
)
```

Internally, `model_run()` performs:

1. Call `model_steady_state()` to compute $\mathbf{C}^*$ from mean climate and inputs.
2. Initialise pools from $\mathbf{C}^*$ (or from the hybrid approach for Q-model).
3. Loop over years (and months for RothC), calling `compute_xi()` and `model_step()` at each timestep.
4. Write annual pool states and respiration to output arrays.

`model_steady_state()` — computes the analytical equilibrium. For Yasso and RothC, this is a matrix inversion $\mathbf{C}^* = -\mathbf{A}^{-1}\mathbf{b}$. For the Q-model, it performs the numerical summation of remaining fractions over a truncated age series.

`model_step()` — advances the pool state by one timestep Δt :

Model	Δt	Core operation
Yasso07/15/20	1 yr	Build $\mathbf{A}$, compute matrix exponential $e^{\mathbf{A}}$, update $\mathbf{C}$
RothC	1/12 yr	Exponential decay per pool, partition decomposed $\mathbf{C}$ via clay ratio x
Q-model	1 yr	Update cumulative $\tilde{\alpha}$ per cohort, compute remaining fraction, prune old cohorts

`compute_xi()` — evaluates the environmental modifiers from climate data and static site properties. Called internally by `model_step()` (or once per year for annual models, 12 times for RothC).

R wrapper layer

The R wrapper for each model handles:

- **Climate mapping:** calls `map_climate_<model>()` once per plot to produce the climate data frame expected by that model. Yasso07/15 use `map_climate_yasso07()` (annual mean + amplitude); Yasso20 and RothC share `map_climate_monthly()` (full monthly series).
- **Input mapping:** calls `map_inputs_<model>()` once per plot to aggregate the six raw cohorts into model-specific litter arrays. Yasso07, Yasso15 and Yasso20 share the same input structure via `map_inputs_yasso07()`.
- **Parameter mapping:** translates the named BayesianTools parameter vector into the positional double-precision vector expected by Fortran.
- **Output formatting:** reshapes the flat Fortran output arrays into named R structures for likelihood evaluation and diagnostics.

All models are run plot-by-plot with a consistent `lapply` $\rightarrow$ `rbind` pattern. Climate and input mapping are performed once per plot, outside the MCMC loop:

```
# --- Once, before calibration ---
plots      <- unique(input_raw$plot_id)
Yasso07_climate <- map_climate_yasso07(input_raw)  # annual: mean, amplitude, precip
monthly_climate <- map_climate_monthly(input_raw) # monthly: shared by Yasso20 + RothC
Yasso07_inputs  <- map_inputs_yasso07(input_raw)  # shared by Yasso07, Yasso15, Yasso20
RothC_inputs    <- map_inputs_rothc(input_raw, site_raw)

# --- Per-plot simulation (inside likelihood, called each MCMC proposal) ---
result <- do.call(rbind, lapply(plots, function(pid) {

  clim  <- Yasso07_climate[Yasso07_climate$plot_id == pid, ]
```



```

inputs <- Yasso07_inputs[Yasso07_inputs$plot_id == pid, ]

# Stage 1: Xi time series from plot-specific climate
xi <- compute_xi_yasso07(clim, params)

# Stage 2: Steady-state initialisation from plot-specific means
C_init <- yasso07_steady_state(
  params = params,
  nwl_mean = colMeans(inputs[, c("nwl_A", "nwl_W", "nwl_E", "nwl_N")]),
  fwl_mean = colMeans(inputs[, c("fwl_A", "fwl_W", "fwl_E", "fwl_N")]),
  cwl_mean = colMeans(inputs[, c("cwl_A", "cwl_W", "cwl_E", "cwl_N")]),
  xi_mean = mean(xi)
)

# Stage 3: Transient simulation - single Fortran call per plot
out <- yasso07_run(inputs, params, C_init, xi)

cbind(plot_id = pid, out)
}))

```

What stays in R vs. Fortran

Component	Language	Rationale
Climate mapping (<code>map_climate_*</code>)	R	Called once per plot; benefits from R data wrangling
Input mapping (<code>map_inputs_*</code>)	R	Called once per plot; benefits from R data wrangling
ξ_k — modifier computation	Fortran	Called millions of times inside MCMC loop
f_{d_k} — decomposition step	Fortran	Innermost loop; matrix operations
Steady-state solver	Fortran	Matrix inversion, numerical integration
Full simulation loop	Fortran	Avoids per-timestep R↔Fortran overhead
Parameter mapping	R	String/name handling; negligible cost
Likelihood evaluation	R	Interface with BayesianTools; simple arithmetic
I/O, aggregation, diagnostics	R	Flexibility; not performance-critical

4.7 Yasso Family: Three-Stage Call Pattern

All three Yasso versions (07, 15, 20) follow the same three-stage execution pattern in R. The differences are confined to Stage 1 (Xi computation) and the climate object passed to each stage.

Climate and input mapping functions

Model	Climate function	Input function
Yasso07	<code>map_climate_yasso07()</code> → annual mean, amplitude, precip	<code>map_inputs_yasso07()</code>
Yasso15	<code>map_climate_yasso07()</code> (reused)	<code>map_inputs_yasso07()</code> (reused)
Yasso20	<code>map_climate_monthly()</code> (shared with RothC)	<code>map_inputs_yasso07()</code> (reused)

All three versions share the same litter input structure: six raw cohorts aggregated to three litter types (nwl, fwl, cwl) with AWEN fractions preserved.

Stage 1 — Xi: climate modifier time series

The key structural difference among the three versions is in how ξ is computed:

Model	Function	Climate resolution	Pool-group specificity
Yasso07	<code>compute_xi_yasso07()</code>	Annual (mean + amplitude → 4-point Gaussian)	Single ξ for all pools
Yasso15	<code>compute_xi_yasso15()</code>	Annual (mean + amplitude → 4-point Gaussian)	Three ξ : AWE, N, H
Yasso20	<code>compute_xi_yasso20()</code>	Monthly (exact 12-month average)	Three ξ : AWE, N, H

Stage 2 — Steady state: initial pool conditions C*

Steady state is computed from plot-specific mean climate and mean litter inputs. For Yasso20, a synthetic 12-row `mean_clim` data frame is constructed from the plot's monthly climatology:

```
# Yasso07 / Yasso15
C_init <- yassoXX_steady_state(
  params = params,
  nwl_mean = colMeans(inputs[, c("nwl_A", "nwl_W", "nwl_E", "nwl_N")]),
  fwl_mean = colMeans(inputs[, c("fwl_A", "fwl_W", "fwl_E", "fwl_N")]),
  cwl_mean = colMeans(inputs[, c("cwl_A", "cwl_W", "cwl_E", "cwl_N")]),
  xi_mean = mean(xi) # or list(xi_awe, xi_n, xi_h) for Yasso15
)

# Yasso20 - requires a synthetic single-year mean climate data frame
# precip must be monthly mean (NOT divided by 12); the function sums
# internally over 12 months to recover the annual total
mean_clim <- data.frame(
  year = 9999L,
  month = 1:12,
  temp_air = tapply(clim$temp_air, clim$month, mean, na.rm = TRUE),
  precip = mean(clim$precip) # monthly mean ~50 mm; sums to ~600 mm/yr internally
)
C_init <- yasso20_steady_state(mean_clim, params, nwl_mean, fwl_mean, cwl_mean)
```

Stage 3 — Transient simulation

All three versions return a flat data frame with columns `year` | `A` | `W` | `E` | `N` | `H` | `total_soc` | `respiration`.

```
out_07 <- yasso07_run(inputs_df,    params, C_init, xi_array)
out_15 <- yasso15_run(inputs_df,    params, C_init, xi_arrays, precip, leac)
out_20 <- yasso20_run(climate_df,   inputs_df, params, C_init, leac)
#                                     ~~~~~~
#                                     monthly_climate - xi_arrays and precip derived internally
```

5. Calibration Parameter Comparison

This section documents the calibration parameter vectors for all four models, their grouping by functional role, and the structural equivalences and differences across models.

5.1 Cross-model parameter map

The table below classifies calibration parameters by functional group. Each cell lists the parameter symbol(s) and count. Dashes indicate that the corresponding process is absent or hardcoded in that model.

Table 15: Calibration parameters by functional group across all four models. Numbers in parentheses give the count of free parameters in that group.

Group	Yasso07	Yasso15	Yasso20	RothC
Decomposition rates	$\alpha_{A,W,E,N,H}$ (5)	$\alpha_{A,W,E,N,H}$ (5)	$\alpha_{A,W,E,N,H}$ (5)	$k_{\text{DPM,RPM,Bio,Hum}}$ (4)
Temperature response	β_1, β_2 (2)	$\beta_1, \beta_2;$ $\beta_{N1}, \beta_{N2};$ β_{H1}, β_{H2} (6)	same as Y15 (6)	a_T, b_T, c_T (3)
Moisture / precipitation response	γ (1)	$\gamma, \gamma_N, \gamma_H$ (3)	$\gamma, \gamma_N, \gamma_H$ (3)	r_{SMD} (1)
Inter-pool transfers	p_{ij} (12)	p_{ij} (12)	p_{ij} (12)	$f_{\text{bio}}, f_{\text{hum}}$ (2)
Humification	p_H (1)	p_H (1)	p_H (1)	via $f_{\text{bio}}, f_{\text{hum}}$
Size / physical modifier	δ_1, δ_2, r (3)	δ_1, δ_2, r (3)	θ_1, θ_2, r (3)	— (absent)
Leaching	— (absent)	w (1)	w (1)	— (absent)
Total	24	31	31	10

Structural notes. The Yasso family has no edaphic modifiers (clay, depth) as free parameters; these enter only through fixed litter-size correction terms.

Regarding inter-pool transfers and humification, RothC and Yasso follow quite different philosophies. In RothC, all four active pools (DPM, RPM, Bio, Hum) decompose and their products are routed to the same two destinations — microbial biomass (Bio) and humus (Hum) — in fixed proportions governed by f_{bio} and f_{hum} . The source pool identity is irrelevant: a unit of carbon leaving DPM and a unit leaving Hum are partitioned identically. This makes the transfer structure simple and symmetric.

In Yasso, the four litter-chemistry pools (A, W, E, N) exchange carbon *with each other* before anything reaches the humus pool H. Each pool j has its own set of transfer fractions p_{ij} controlling what fraction of its decomposed material flows into each other pool i . For example, decomposed W-pool carbon might partially re-enter the A pool, partially enter E, and so on. These 12 fractions ($p_{WA}, p_{EA}, \dots, p_{EN}$) together define the full AWEN transfer matrix, and they are all free calibration parameters. On top of this lateral exchange, a separate parameter p_H controls the fraction of decomposed carbon from *any* AWEN pool that is additionally routed to the humus pool H — analogous in spirit to RothC’s f_{hum} , but applied uniformly across all source pools. The net effect is that Yasso’s pool dynamics are governed by a full 5×5 matrix, whereas RothC’s are governed by a simpler one-directional cascade with a shared partitioning step at the end. RothC’s f_{bio} and f_{hum} therefore simultaneously play the role of both the inter-pool transfer coefficients *and* the humification fraction, compressed into just two parameters.

5.2 Yasso family parameter table

The three Yasso versions share the same pool structure and transfer matrix. Differences are confined to the climate response parameterisation (Yasso15/20 add pool-group-specific modifiers) and the size modifier

functional form (Yasso20 uses a quadratic rather than exponential formulation).

`\begin{table}[H] \caption{Yasso family calibration parameters.`

Gray

: shared by Y07, Y15, Y20.

Green

: introduced in Y15, retained in Y20.

Amber

: Y20-specific (replaces Y07/Y15 formulation). All α rates are positive; the Fortran negates them internally on the diagonal.}

Group	Symbol	Default	Units	Description
<i>Decomposition rates — shared by Y07, Y15, Y20</i>				
	α_A	0.66	yr ⁻¹	Base decomp. rate, acid-soluble (A) pool
	α_W	4.30	yr ⁻¹	Base decomp. rate, water-soluble (W) pool
	α_E	0.35	yr ⁻¹	Base decomp. rate, ethanol-soluble (E) pool
	α_N	0.22	yr ⁻¹	Base decomp. rate, non-soluble/lignin (N) pool
	α_H	0.0033	yr ⁻¹	Base decomp. rate, humus (H) pool
<i>Inter-pool transfers — shared by Y07, Y15, Y20</i>				
	p_{WA}, p_{EA}, p_{NA}	—	—	Fractions of W, E, N decomposition entering A
	p_{AW}, p_{EW}, p_{NW}	—	—	Fractions of A, E, N decomposition entering W
	p_{AE}, p_{WE}, p_{NE}	—	—	Fractions of A, W, N decomposition entering E
	p_{AN}, p_{WN}, p_{EN}	—	—	Fractions of A, W, E decomposition entering N
	p_H	0.04 / 0.0042*	—	Fraction of AWEN decomposition entering H
<i>Temperature response (AWE pools) — shared by Y07, Y15, Y20</i>				
	β_1	0.096	°C ⁻¹	Linear temperature coefficient
	β_2	-0.0014	°C ⁻²	Quadratic temperature coefficient
<i>Temperature response (N pool) — Y15 and Y20 only</i>				
	β_{N1}		°C ⁻¹	Linear temperature coefficient, N pool
	β_{N2}		°C ⁻²	Quadratic temperature coefficient, N pool
<i>Temperature response (H pool) — Y15 and Y20 only</i>				
	β_{H1}		°C ⁻¹	Linear temperature coefficient, H pool
	β_{H2}		°C ⁻²	Quadratic temperature coefficient, H pool
<i>Precipitation response (AWE pools) — shared by Y07, Y15, Y20</i>				
	γ	-1.21	m ⁻¹	Shape of precipitation response, $1 - e^{\gamma P/1000}$
<i>Precipitation response (N, H pools) — Y15 and Y20 only</i>				
	γ_N		m ⁻¹	Precipitation response shape, N pool
	γ_H		m ⁻¹	Precipitation response shape, H pool
<i>Leaching — Y15 and Y20 only</i>				
	w		mm ⁻¹	Precipitation-driven leaching rate for AWEN pools
<i>Size modifier — Y07 and Y15</i>				
	δ_1, δ_2	-1.7, 0.86	—	Exponent parameters, $\exp(-\delta_1 d^{\delta_2} r)$
	r	-0.306	—	Scaling parameter (shared with Y20)
<i>Size modifier — Y20 only (replaces δ_1, δ_2)</i>				
	θ_1, θ_2		cm ⁻¹ , cm ⁻²	Quadratic size modifier, $(1 + \theta_1 d + \theta_2 d^2)^{- r }$

* $p_H = 0.04$ for Y07/Y15; $p_H = 0.0042$ for Y20 — a 10-fold reduction.

\end{table}

5.3 RothC parameter table

RothC's parameter vector is deliberately leaner than the Yasso family, reflecting its simpler pool structure (no lateral AWEN transfers, no woody litter size effect). The 10 proposed calibration parameters are listed

below, with their functional analog in the Yasso family where one exists.

Table 16: RothC calibration parameters. The rightmost column identifies the closest functional analog in the Yasso family where applicable.

Group	Symbol	Default	Description	Yasso analog
<i>Decomposition rates</i>				
	k_{DPM}	10.0	DPM decomp. rate (yr^{-1})	α_A or α_W
	k_{RPM}	0.30	RPM decomp. rate (yr^{-1})	α_N
	k_{Bio}	0.66	BIO decomp. rate (yr^{-1})	α_E
	k_{Hum}	0.02	HUM decomp. rate (yr^{-1})	α_H
<i>Temperature response</i>				
	a_T	47.91	Numerator of Arrhenius modifier	β_1, β_2 (different form)
	b_T	106.06	Activation energy proxy	β_1, β_2 (different form)
	c_T	18.27	Temperature offset ($^{\circ}\text{C}$)	β_1, β_2 (different form)
<i>Moisture response</i>				
	r_{SMD}	0.444	Breakpoint of piecewise linear TSMD response, as fraction of TSMD_{max}	γ (different driver)
<i>Inter-pool transfers and humification</i>				
	f_{bio}	0.46	Fraction of decomposed C entering BIO	p_H, p_{ij} (combined role)
	f_{hum}	0.54	Fraction of decomposed C entering HUM	p_H, p_{ij} (combined role)
<i>Hardcoded — not calibrated</i>				
	evap fraction	0.75	Coefficient on open-pan evaporation in TSMD accumulation	—
	r_{bare}	0.556	Bare soil TSMD limit as fraction of TSMD_{max}	—
	ξ_{min}	0.20	Floor on moisture modifier under dry conditions	—
	clay coefficients		Pedotransfer function for TSMD capacity	—

Note on f_{bio} and f_{hum} . In the standard RothC formulation these fractions are derived from clay content via $x = 1.67(1.85 + 1.60 e^{-0.0786\% \text{clay}})$, with $f_{\text{bio}} = 0.46/(x + 1)$ and $f_{\text{hum}} = 0.54/(x + 1)$. By freeing the numerators 0.46 and 0.54 as direct calibration parameters (while retaining the clay-dependent denominator), we preserve the soil-texture sensitivity of the original formulation while giving the Bayesian sampler a handle on the bio–hum split. This is the closest structural analog to Yasso’s p_H combined with the inter-pool transfer fractions.

6. Calibration Engine Architecture

6.1 Design Principle

The Bayesian calibration framework is split into two layers: a **model-agnostic engine** shared by all models, and a thin **model-specific interface** that each model script implements. The engine handles everything that does not depend on soil carbon dynamics; the interface provides only the three functions that do.

This design means that changes to convergence diagnostics, output management, logging, or MCMC settings are made once and propagate to all five model calibrations automatically. It also guarantees that all models are evaluated under identical statistical assumptions, which is a prerequisite for valid structural uncertainty quantification.

6.2 Engine Components

The following components are fully model-agnostic and implemented once in `calibration_engine.R`:

Transform builder. Constructs `to_original()`, `to_unconstrained()`, and the log-Jacobian function automatically from the model's `param_spec` declaration. See Section 6.4 for details.

Prior predictive matching. The procedure for choosing the prior width σ_{ppm} is identical across models: draw samples from a Gaussian centred on `best_x`, transform to physical space, run one plot forward, compute the coefficient of variation of predicted SOC, and find the smallest σ whose prior predictive CV meets or exceeds the observed CV. The engine requires only a single-plot forward run function from the model interface. On synthetic data with few unique plots the matching may not converge; the engine warns and defaults to $\sigma_{\text{ppm}} = 1.0$.

Likelihood factory (`make_likelihood()`). Returns a byte-compiled likelihood function with all model callables and pre-computed data structures captured in its closure. Receives the three model interface functions, the pre-split data structures, and the transform functions. The returned function is passed to `run_mcmc_chains()`.

MCMC orchestration (`run_mcmc_chains()`). Chain initialisation, the DEzs sampler loop, per-chain log files, the evaluation counter, and elapsed-time tracking. Receives the likelihood function, prior object, and both `mcmc_settings` and `run_config`. The outer chain loop (`lapply` on Mac, `mclapply` on Roidhu) and the inner parallel plot loop (`mclapply` with `CORES_PER_CHAIN`) are both managed here.

Convergence diagnostics (`run_diagnostics()`). Gelman–Rubin $\hat{R}$ via `gelman.diag`, effective sample size via `effectiveSize`, posterior quantile tables, trace PDFs, and prior-vs-posterior marginal PDFs. Receives `mcmc_settings` as a separate argument (not via `run_config`) to access iteration counts. All diagnostics operate on raw MCMC samples and parameter names; they have no knowledge of what model produced them.

Output management (`save_results()`). Serialises three files to `DIR_RUNS`: the raw BayesianTools chain objects (`MODEL_chains_RUNID.rds`), the physical-space posterior samples matrix `all_phys` received from `run_diagnostics()` (`MODEL_posterior_RUNID.rds`), and run metadata including convergence summaries (`MODEL_metadata_RUNID.rds`). Receives `mcmc_settings` for iteration counts.

Input serialisation (`save_inputs()`). Serialises all pre-processed data structures that entered the likelihood to `DIR_INPUTS` as `MODEL_inputs_RUNID.rds`. Called after prior predictive matching so that σ_{ppm} is included in the saved object alongside `climate_by_plot`, `inputs_by_plot`, `litter_means`, `obs_meta`, `plots`, `plots_real`, `free_defaults`, and `best_x`. Serves two purposes: enabling post-hoc analysis (posterior predictive checks, residual diagnostics) without re-sourcing the calibration script; and providing a validation record that the input pipeline produced the expected structures, independently verifiable against the raw data.

Startup checks. Mandatory round-trip validation (physical $\rightarrow$ unconstrained $\rightarrow$ physical to machine precision) and likelihood finiteness check at default parameters, executed before sampling begins.

Configuration structure. `run_config` (output directories, model name, run ID, cores) and `mcmc_settings` (chain count, iteration counts, burn-in, log interval) are both assembled in Section~0 of each model script at startup, before any data loading or computation. The only field added later is `run_config$n_plots`, appended at the end of Section~3 once the plots vector is known. Both objects are passed explicitly to every engine function that needs them.

6.3 Model Interface

Each model script passes three functions to the engine as arguments to `make_likelihood()`. These are not new functions written for calibration — they are the same R wrapper functions already implemented for forward model runs, validated against the reference Fortran implementations, and used independently for testing and simulation.

Table 17: The three functions constituting the model–engine interface.

Function	Signature	Role
<code>compute_xi(clim, params)</code>	Climate data frame + parameter vector	Returns environmental modifier time series $\xi(t)$; called once per plot per likelihood evaluation. Must live inside the sampler because climate sensitivity parameters $(\beta_1, \beta_2, \gamma)$ are free calibration parameters.
<code>steady_state(params, litter_means, xi_mean)</code>	Parameter vector + mean litter inputs + mean ξ	Returns initial pool vector $\mathbf{C}^*$; called once per plot per likelihood evaluation.
<code>run_model(inputs, params, C_init, xi)</code>	Input data frame + parameters + $\mathbf{C}^* + \xi(t)$	Runs full transient simulation; returns SOC trajectory; single Fortran call per plot.

For example, the Yasso07 model script passes `compute_xi_yasso07`, `yasso07_steady_state`, and `yasso07_run` to `make_likelihood()`, which captures them in the likelihood closure. The same functions are used for standalone forward runs and validation against the reference implementation.

```
# Forward run -- direct function call, fixed parameters
out <- yasso07_run(inputs_df,
  params    = YASSO07_DEFAULT_PARAMS,
  C_init    = C_init,
  xi_array  = xi_array)

# Calibration -- same functions injected into likelihood factory,
# which is then passed to the MCMC engine
ll_fn <- make_likelihood(
  compute_xi    = compute_xi_yasso07, # same function as forward run
  steady_state  = yasso07_steady_state, # same function as forward run
  run_model     = yasso07_run,          # same function as forward run
  to_original   = to_original,
  log_jacobian  = log_jacobian,
  assemble_params = assemble_model_params,
  plots        = plots,
```

```

climate_by_plot = climate_by_plot,
inputs_by_plot  = inputs_by_plot,
litter_means    = litter_means,
obs_meta        = obs_meta,
n_cores         = CORES_PER_CHAIN
)

chain_results <- run_mcmc_chains(
  ll_fn          = ll_fn,
  prior          = prior,
  best_x         = best_x,
  param_names    = FREE_NAMES,
  mcmc_settings  = mcmc_settings,
  run_config     = run_config
)

```

The only requirement the engine imposes on the three interface functions is that they respect the calling conventions listed in the table above. Any function that runs correctly in forward mode already satisfies this requirement. There is no translation layer, no reformatting of inputs or outputs, and no risk of the calibration exercising a different code path than the validated forward run.

6.4 Parameter Transformation Framework

Transformations between constrained physical space and unconstrained sampling space are generated automatically from a **parameter specification table** defined in each model script. Rather than writing model-specific `to_original` and `to_unconstrained` functions, the engine constructs these from a list of typed parameter groups.

Three primitive transform types cover all parameters across all four models:

Table 18: Primitive transformation types and their application across models.

Type	Constraint	Transform	Applied to
stick_break	$\sum p_i \leq B, p_i \geq 0$	Sequential logit fractions of remaining budget	Yasso AWEN inter-pool transfer fractions (p_{ij} , 12 per model)
log	$\theta > 0$	$x = \log \theta$	Variance and scaling terms (σ_{obs} , σ_{init} , σ_{input})
unconstrained	$\theta \in \mathbb{R}$	Identity	Temperature and precipitation response parameters (β_i , γ); size modifier parameters (δ_i , r)
logit	$\theta \in (0, 1)$	$x = \log \frac{\theta}{1-\theta}$	Single proportions; RothC f_{bio} , f_{hum}

Fixed vs. free parameters. Decomposition rate constants ($\alpha_{A,W,E,N,H}$ for Yasso; $k_{\text{DPM,RPM,Bio,Hum}}$ for RothC) and the humification fraction p_H are held fixed at published values throughout MCMC. These encode each model’s pool chemical identity and are independently constrained by experimental data; calibrating them jointly with transfer fractions is not identifiable and would blur the structural differences the intercomparison is designed to measure. The free parameters — transfer fractions, climate sensitivity, size modifiers, and variance and scaling terms — govern climate response, partitioning, and input uncertainty, the quantities most relevant to Finnish climate projections.

The Yasso07 parameter specification takes the form:

```

# Budget for stick-breaking: fraction not going to humus
B <- 1.0 - fixed_rates["p_H"]

param_spec <- list(
  # Inter-pool transfer fractions -- one stick-breaking group per source pool
  list(names = c("p_AW", "p_AE", "p_AN"), type = "stick_break", budget = B),
  list(names = c("p_WA", "p_WE", "p_WN"), type = "stick_break", budget = B),
  list(names = c("p_EA", "p_EW", "p_EN"), type = "stick_break", budget = B),
  list(names = c("p_NA", "p_NW", "p_NE"), type = "stick_break", budget = B),
  # Climate sensitivity -- unconstrained (can be negative)
  list(names = c("beta1", "beta2", "gamma"), type = "unconstrained"),
  # Size modifiers -- unconstrained
  list(names = c("delta1", "delta2", "r"), type = "unconstrained"),
  # Variance and scaling terms -- strictly positive
  list(names = c("sigma_obs", "sigma_init", "sigma_input"), type = "log")
)

```

The engine builds `to_original`, `to_unconstrained`, and the Jacobian from this spec. The Jacobian is assembled by summing contributions per group: stick-breaking groups contribute the sequential logit volume correction; log-transformed parameters contribute $\log \theta$ each; unconstrained parameters contribute zero.

Test mode replication. The `N_PLOTS_TEST` parameter replicates plot IDs in the `plots` vector to simulate production wallclock cost without requiring the full dataset:

```

# plots is a vector of plot IDs passed to the likelihood -- either the full
# production set or N_PLOTS_TEST replications of plots_real for timing tests.
plots <- if (!is.na(N_PLOTS_TEST)) {
  rep(plots_real, length.out = N_PLOTS_TEST)
} else {
  plots_real
}

```

The replication is a lightweight index trick: `climate_by_plot` and `inputs_by_plot` retain only the unique real plots; the likelihood evaluates the same plots repeatedly. This gives a realistic wallclock estimate for production scaling but provides no new statistical information. **Convergence diagnostics from replicated test runs are meaningless and should be ignored** — the sampler sees repeated identical data and cannot distinguish structural from sampling uncertainty.

6.5 Known Validation Items

The following issues were identified during implementation and should be verified before the production run on real data:

ξ_{mean} timing. The analytical steady-state uses `mean(xi_array)` computed over the full simulation time series, including years after the last observation. For a plot measured in 1997–2006, this includes post-2006 climate. Strictly, the steady-state should use pre-measurement climate only. This inconsistency is absorbed into `sigma_init` and may not be detectable, but introduces a systematic bias in plots with strong climate trends.

Jacobian passthrough in BayesianTools. The log Jacobian is added inside the likelihood rather than the prior, which keeps the BayesianTools prior object clean. This assumes `createPrior()` passes the custom density function through unchanged and does not apply additional normalisation. This assumption should be verified against the BayesianTools source for the version in use.

Year alignment between Fortran output and `obs_meta$idx`. The observation year indices are computed as `match(obs_plot$year, clim$year)`, which assumes the Fortran model returns one row per year

in the same order as the climate input data frame. If the Fortran wrapper reorders or drops years, `meta$idx` points to wrong rows without triggering any guard. This should be validated by running a forward model call and checking that `run_out$total_soc[meta$idx]` corresponds to the correct years.

6.6 Prior Specification

All free parameters share a common Gaussian prior in unconstrained sampling space:

$$x_i \sim \mathcal{N}(\text{best_x}_i, \sigma_{\text{ppm}}^2) \quad \forall i$$

where `best_xi` is the published default value of parameter i transformed to unconstrained space, and σ_{ppm} is a single scalar width shared across all parameters. The value of σ_{ppm} is chosen separately for each model by prior predictive matching: it is set to the smallest value such that the coefficient of variation of SOC predictions drawn from the prior matches the observed CV across plots. This normalises prior coverage in prediction space rather than parameter space, ensuring that structural differences in posterior predictions reflect model structure rather than differences in prior informativeness. A model requiring a wider σ_{ppm} to reach the observed CV is collectively less sensitive to its free parameters in SOC space — itself a structural finding. Fixed parameters — decomposition rate constants and humification fraction p_H — are held at published values and do not enter the prior.

Table 19: Prior specification for all calibrated parameters. All parameters share the same Gaussian prior form in unconstrained space; the transform determines the implied prior shape in physical space. σ_{ppm} is set by prior predictive matching and differs across models.

Parameter class	Parameters	Transform	Prior (unconstrained)
Inter-pool transfer fractions	p_{ij} (12, Yasso); $f_{\text{bio}}, f_{\text{hum}}$ (RothC)	stick-breaking / logit	$\mathcal{N}(\text{best_x}, \sigma_{\text{ppm}}^2)$
Climate sensitivity	β_1, β_2, γ (Yasso); $a_T, b_T, c_T, r_{\text{SMD}}$ (RothC)	unconstrained	$\mathcal{N}(\text{best_x}, \sigma_{\text{ppm}}^2)$
Size modifiers	δ_1, δ_2, r (Yasso only)	unconstrained	$\mathcal{N}(\text{best_x}, \sigma_{\text{ppm}}^2)$
Variance & scaling terms	$\sigma_{\text{obs}}, \sigma_{\text{init}}, \sigma_{\text{input}}$	log	$\mathcal{N}(\text{best_x}, \sigma_{\text{ppm}}^2)$
<i>Fixed — not calibrated</i>	$\alpha_{A,W,E,N,H}; k_{\text{DPM,RPM,Bio,Hum}}; p_H$	—	fixed at published values

Variance and scaling terms. Three nuisance parameters are jointly calibrated with the structural parameters. σ_{obs} is the standard deviation of residuals between predicted and observed SOC at measurement time, capturing measurement error, within-plot spatial heterogeneity, and any model structural error that presents as noise. σ_{init} captures uncertainty in the analytical steady-state initialisation: real plots are not at equilibrium, and the mean climate used in $\mathbf{C}^* = -\mathbf{A}^{-1}\mathbf{b}$ is itself an estimate. σ_{input} is a global multiplicative scaling factor on all litter inputs, $\mathbf{J}_{\text{scaled}}(t) = \sigma_{\text{input}} \cdot \mathbf{J}(t)$, which accounts for systematic bias in the allometric-equation-derived litter estimates. A posterior value below 1 indicates the allometric equations overestimate inputs for Finnish forests; above 1 indicates underestimation. Because decomposition rates are fixed, σ_{input} is not strongly confounded with rates and is expected to be identifiable from the SOC data.

6.7 Architecture Overview

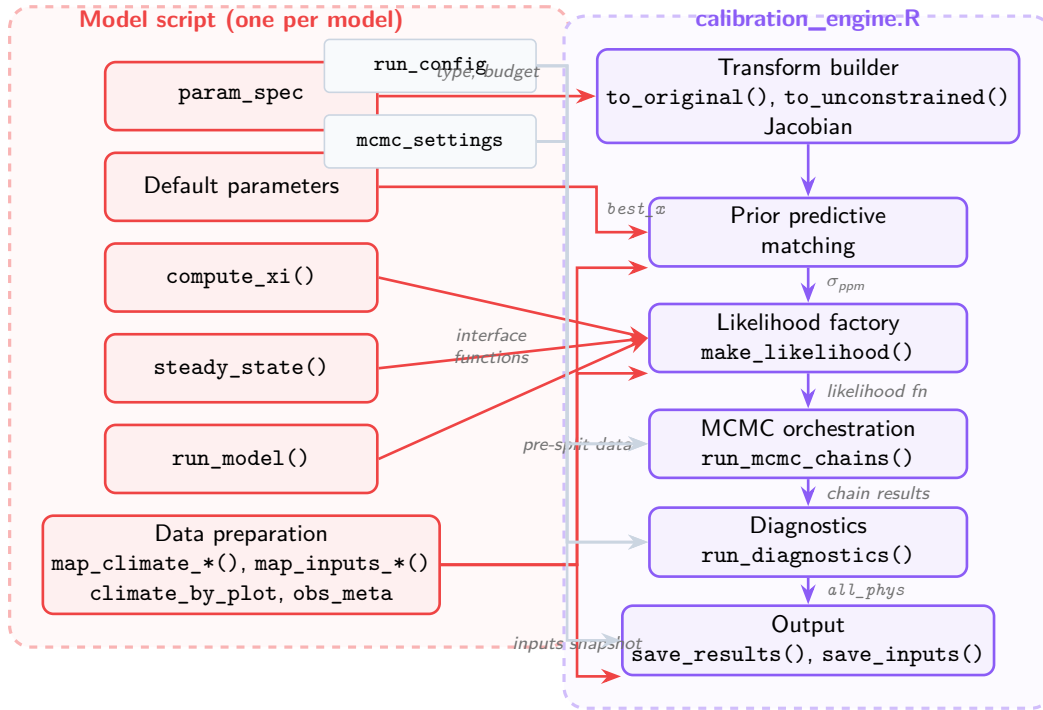


Figure 3: High-level structure of the calibration framework. The **model script** (red) contains everything model-specific: a typed parameter specification, published default parameters, the three interface functions (`compute_xi`, `steady_state`, `run_model`), model-specific data preparation (input mapping functions, pre-splitting into per-plot lookup structures), and Section 0 configuration (`run_config`, `mcmc_settings`). The **engine** (`calibration_engine.R`, purple) is fully model-agnostic: it constructs transformation functions and the Jacobian from the parameter specification, runs prior predictive matching to set σ_{ppm} , assembles the generic likelihood, orchestrates MCMC sampling, computes diagnostics, and serialises outputs. All data structures produced by the model script’s preparation step (`climate_by_plot`, `obs_meta`, etc.) are passed into the engine via `make_likelihood()`’s closure arguments — they are not a separate architectural layer.